8/10/26

EXPERIMENT-14

IMPLEMENT PACKET SNIFFING USING RAW SOCKETS

AIM:

To write a program using RAW sockets to capture & display network packets passing through the network interface.

PROCEDURE:

→ Start the program.
→ Create a RAW socket using socket(AF_PACKET, SOCK_RAW, ...).
→ Bind the socket to the required network interface.
→ Continously receive packets from the network.
→ Extract the Ethernet header from each packet.
→ Identify the Ethernet header from each packet.
→ Identify the protocol type, such as, IPV4, ARP or IP6.
→ IPV4, extract the source & destination IP addresses.
→ Display the packet information.
→ Repeat until the user terminates the program.
→ close the socket.

PROGRAM:

```
import socket
import struct

raw_socket = socket.socket(socket.AF_PACKET, socket.
                           SOCK_RAW, socket.ntohs(3))

try:
    while TRUE:
        packet, address = raw_socket.recvfrom(65535)
        ethernet_header = packet[1:4]
        dest_mac, src_mac, protocol = struct.unpack(
            "!6s6sH", ethernet_header)
```

```
print("Source MAC : ", src_mac)
print("Destination MAC: ", dest_mac)
print("Protocol : ", hex(protocol))

if protocol == 0x800:
    ip_header = packet[14:34]

    src_ip = inet_ntoa(src_ip)
    dest_ip = socket.inet_ntoa(src_ip)

    print("Source IP : ", src_ip)
    print("Destination Ip : ", dest_ip)
    print("TTL : ", ttl)
    print("IP Protocol : ", protocol)

elif protocol == 0x0806:
    print("Packet type : ARP")

elif protocol == 0x86DD:
    print("Packet type ", ": IPV6")

else:
    print("Packet type : Other")

except KeyboardInterrupt:
    print("\n Packet sniffing stopped")

finally:
    raw_socket.close()
```

RESULT:

Thus, a packet-sniffing program using RAW sockets was implemented. The program captures network packets and displays important information such as source MAC, destination MAC addresses, protocol, source IP & destination IP address & TTL.

SAMPLE OUTPUT:

Packet sniffer started...

Waiting for packets....

Source MAC: 3C:52:82:11:22:33

Destination MAC: ff:ff:ff:ff:ff:ff

Protocol: 0x806

Packet Type: ARP

Source MAC: 3C:52:82:11:22:33

Destination MAC: 84:16:f9:44:55:66

Protocol: 0x800

Source IP: 192.168.1.10

Destination IP: 192.168.1.1

TTL: 64

IP Protocol: 6

Source MAC: 84:16:f9:44:55:66

Destination MAC: 3C:52:82:11:22:33

Protocol: 0x800

Source IP: 192.168.1.1

Destination IP: 192.168.1.10

TTL: 64

IP Protocol: 6